HYBRID COMPUTING

Mata Kuliah Parallel & Distributed Computing

(Section 6: Hybrid Computing)



Disusun oleh

Parhan 247006111183

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS SILIWANGI

2026

DAFTAR ISI

DAFTAR ISI 2
BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Lingkungan dan Parameter 4
1.4 Ringkasan Laporan 4
Spesifikasi Lingkungan Eksperimen 5
Status Bagian Tugas 5
BAB II 6
PEMAHAMAN KONSEP 6
21. Parallel Computing vs Distributed Computing 6
2.2. Pure MPI vs Hybrid MPI + OpenMP 6
2.3. Thread, ProcessPool, dan GIL 7
2.4. Skenario Deadlock pada MPI + Threads 7
2.5. Hukum Amdahl 8
BAB III 9
PRAKTIKUM 9
3.1. Eksperimen Hybrid Pipeline 9
3.2. Strong Scaling MPI + ProcessPool 11
3.2.1 Identifikasi dan Perbaikan Error 11
3.2.2 Parameter dan Hasil 13
3.3. Modifikasi Global Word Count 15
3.3.1 Hasil Perbandingan Waktu 16
3.3.2 Top 10 Kata 17
BAB IV 19

DESAIN ARSITEKTUR HYBRID COMPUTING 19
4.1 Kasus yang Dipilih 19
4.2 Arsitektur Tiga Level Paralelisme 19
4.3 Teknologi dan Alasan Pemilihan 20
4.4 Klasifikasi Workload 20
4.5 Tantangan dan Strategi Mitigasi 20
4.6 Alur Kerja Sistem 21
BAB V 22
KESIMPULAN 22
REFERENSI 23
LAMPIRAN - DAFTAR FILE DAN BUKTI YANG DIKUMPULKAN 24

BAB I PENDAHULUAN

1.1 Latar Belakang

Hybrid Computing menggabungkan paralelisme lokal di dalam node dengan komputasi terdistribusi antar-node. Pendekatan ini dibutuhkan ketika satu model paralel saja tidak cukup untuk menangani keterbatasan core, memori, atau kebutuhan distribusi data. Materi kuliah menjelaskan kombinasi threads, process pool atau GPU pada level intra-node dan MPI pada level inter-node sebagai pola utama hybrid computing.

1.2 Tujuan

Laporan ini bertujuan menjelaskan konsep hybrid computing melalui jawaban A1-A5, memvalidasi konsep melalui eksperimen B1-B3, serta merancang arsitektur hybrid untuk kasus analisis citra satelit. Selain itu, laporan mendokumentasikan parameter eksperimen, hasil pengukuran, analisis bottleneck, dan implikasi scaling.

1.3 Lingkungan dan Parameter

Parameter pribadi yang digunakan adalah A = 3 berdasarkan digit terakhir NPM 247006111183. Parameter ini menghasilkan dataset B1 sebanyak 130 file dan SAMPLES_PER_TASK B2 sebesar 230.000.

1.4 Ringkasan Laporan

Laporan ini membahas pemahaman konsep Hybrid Computing, eksperimen Hybrid Pipeline, Strong Scaling MPI + ProcessPool, modifikasi Global Word Count, serta desain arsitektur hybrid untuk analisis citra satelit pemetaan lahan pertanian di Jawa Barat. Parameter pribadi yang digunakan adalah A = 3 berdasarkan digit terakhir NPM 247006111183.

Eksperimen dilakukan pada laptop Windows dengan Intel Core i5-12450H (8 core dan 12 logical processor), RAM 15,65 GB, Windows 11 NT 10.0.26200, dan Python 3.11.9. Implementasi MPI menggunakan MS-MPI dan mpi4py.

Spesifikasi Lingkungan Eksperimen

Komponen	Spesifikasi
CPU	Intel Core i5-12450H, 8 core, 12 logical processor
RAM	15,65 GB
Sistem Operasi	Windows 11 NT 10.0.26200
Python	3.11.9
MPI	MS-MPI + mpi4py 4.1.2
Editor	Visual Studio Code

Status Bagian Tugas

Bagian	Status
A1-A5	Selesai
B1	Selesai
B2	Selesai
B3	Selesai
C	Selesai
Referensi	Selesai

BAB II

PEMAHAMAN KONSEP

2.1. Parallel Computing vs Distributed Computing

Komputasi paralel dan komputasi terdistribusi masing-masing memiliki batas ketika digunakan secara tunggal untuk masalah berskala besar. Komputasi paralel pada satu node dapat memanfaatkan shared memory antar-core, tetapi kapasitasnya dibatasi oleh jumlah core, kapasitas RAM, bandwidth memori, dan sumber daya perangkat pada satu komputer. Ketika ukuran masalah terus bertambah, satu node pada akhirnya tidak lagi menyediakan kapasitas komputasi atau memori yang cukup.

Komputasi terdistribusi memperluas kapasitas dengan menggabungkan beberapa node yang masing-masing mempunyai memori sendiri (distributed memory). Kelemahannya adalah data antar-node harus dikomunikasikan melalui jaringan sehingga menimbulkan latency dan overhead. Karena itu, Hybrid Computing menggabungkan distributed computing antar-node dengan parallelism lokal di dalam node. Contohnya MPI membagi data ke beberapa node, kemudian thread, process pool, atau GPU mengerjakan bagian lokal pada masing-masing node. Pendekatan ini memadukan skalabilitas cluster dengan pemanfaatan sumber daya lokal .

2.2. Pure MPI vs Hybrid MPI + OpenMP

Aspek	Pure MPI: 128 proses	Hybrid: 4 MPI x 32 thread
Memori	Setiap proses memiliki address space sendiri; kebutuhan memori dan data yang direplikasi dapat meningkat.	Lebih banyak data dapat berada dalam memori bersama per node; jumlah endpoint MPI lebih sedikit.
Komunikasi	128 proses menyebabkan lebih banyak pasangan/endpoint komunikasi MPI dan dapat memperbesar overhead.	Hanya 4 proses MPI antar-node; komunikasi antar-node cenderung lebih sedikit, sedangkan komunikasi intra-node memanfaatkan shared memory.
Kelebihan	Model MPI relatif seragam dan mudah dipetakan satu proses per unit kerja.	Potensi pemakaian core dalam satu node lebih efisien dan mengurangi jumlah proses MPI.

Aspek	Pure MPI: 128 proses	Hybrid: 4 MPI x 32 thread
Kekurangan	Pemakaian memori dapat lebih besar dan komunikasi MPI dapat menjadi lebih dominan.	Pemrograman lebih kompleks karena harus mengelola MPI dan threading sekaligus, termasuk sinkronisasi thread.
Kemudahan pemrograman	Lebih sederhana karena satu model paralel utama.	Lebih kompleks karena dua level paralelisme dan sinkronisasi harus dirancang dengan benar.

Pada cluster 4 node x 32 core, kedua pendekatan menyediakan 128 unit eksekusi, tetapi struktur komunikasinya berbeda. Pure MPI mengekspos seluruh 128 proses ke MPI. Hybrid hanya mengekspos 4 rank MPI dan menggunakan 32 thread di tiap node. Dengan demikian, hybrid dapat mengurangi komunikasi antar-node, tetapi programmer harus memperhatikan interaksi MPI dengan thread dan keseimbangan beban.

2.3. Thread, ProcessPool, dan GIL

ThreadPoolExecutor cocok untuk pekerjaan I/O-bound karena thread dapat saling tumpang tindih ketika sebagian thread sedang menunggu operasi input/output. Dalam CPython, Global Interpreter Lock (GIL) membatasi eksekusi bytecode Python secara simultan oleh banyak thread pada interpreter yang sama, sehingga thread tidak menjadi pilihan utama untuk komputasi CPU-bound Python murni. ProcessPoolExecutor menggunakan proses terpisah sehingga pekerjaan CPU-bound dapat memanfaatkan beberapa core dan tidak berbagi GIL yang sama.

Apabila perannya ditukar, yaitu ProcessPool dipakai untuk I/O-bound dan ThreadPool untuk CPU-bound, sistem tidak otomatis menjadi lebih cepat. ProcessPool untuk I/O dapat menambah overhead pembuatan proses dan pertukaran data, sedangkan ThreadPool untuk CPU-bound Python murni biasanya tidak memberikan paralelisme CPU yang sama karena GIL. Oleh sebab itu, pemilihan model harus mengikuti karakter workload, bukan sekadar jumlah worker .

2.4. Skenario Deadlock pada MPI + Threads

Contoh deadlock terjadi ketika Rank 0 mempunyai Thread A yang terlebih dahulu mengambil lock bersama lalu melakukan komunikasi MPI yang bersifat blocking. Pada saat yang sama Rank 1 mempunyai Thread B yang harus memperoleh lock tertentu sebelum

melakukan MPI_Recv. Rank 0 menunggu Rank 1 menerima pesan, sedangkan Rank 1 menunggu lock yang belum dilepas. Akibatnya kedua rank saling menunggu dan program tidak pernah melanjutkan.

Pencegahannya adalah tidak menahan lock ketika melakukan operasi MPI blocking, menetapkan urutan pengambilan lock yang konsisten, membatasi satu thread khusus sebagai pengendali komunikasi MPI bila sesuai dengan desain, dan menggunakan komunikasi non-blocking ketika memang dibutuhkan. Selain itu, sinkronisasi harus dibuat eksplisit sehingga tidak ada siklus ketergantungan antara thread, proses, dan MPI.

2.5. Hukum Amdahl

Parameter pribadi: A = 3, sehingga bagian serial program adalah (5 + 3)% = 8% atau s = 0,08. Bagian paralel adalah 92% atau 0,92. Hukum Amdahl menggunakan rumus S(N) = 1 / (s + (1-s)/N). Efisiensi dihitung dengan E(N) = S(N)/N.

Konfigurasi	N worker	Perhitungan speedup	Speedup teoritis	Efisiensi
4 node x 8 core	32	1 / (0,08 + 0,92/32)	9,1954 x	28,74%
16 node x 8 core	128	1 / (0,08 + 0,92/128)	11,4695 x	8,96%

Batas atas speedup ketika jumlah worker menuju tak hingga adalah 1/0,08 = 12,5x. Artinya, walaupun jumlah sumber daya terus ditambah, bagian serial 8% tetap membatasi percepatan. Ketika worker bertambah dari 32 menjadi 128, speedup teoritis hanya naik dari sekitar 9,20x menjadi 11,47x, bukan empat kali lipat. Efisiensi bahkan turun karena sumber daya tambahan semakin kurang memberikan peningkatan speedup proporsional. Hasil ini menunjukkan bahwa klaim performa meningkat proporsional dengan sumber daya hanya berlaku ketika overhead dan bagian serial cukup kecil.

BAB III

PRAKTIKUM

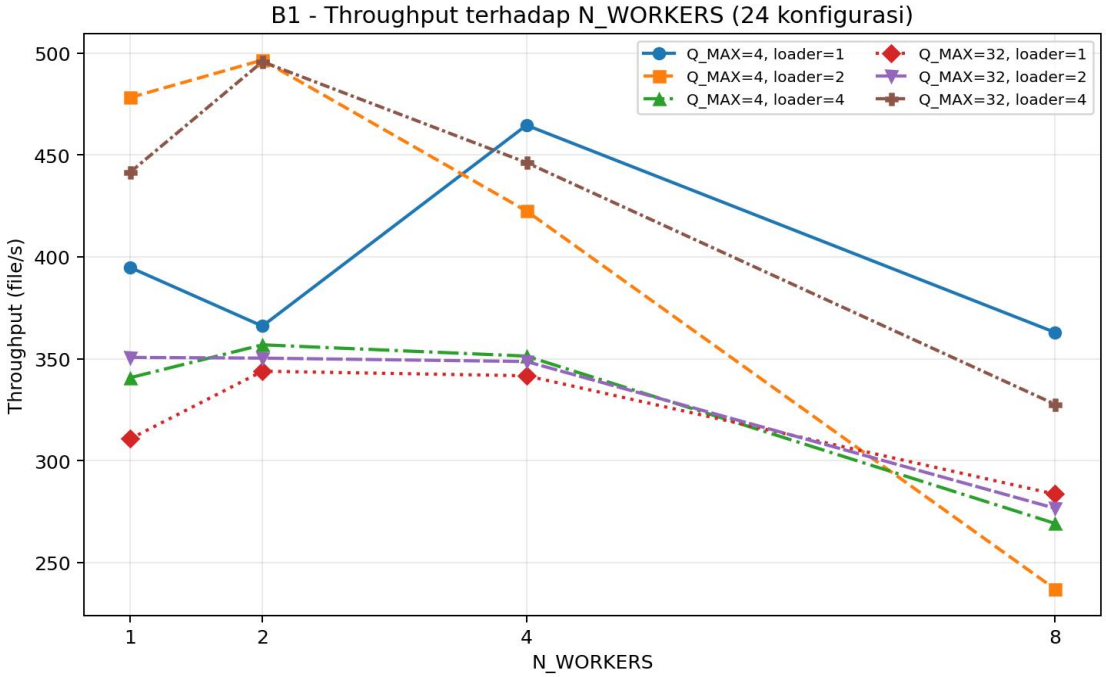
3.1. Eksperimen Hybrid Pipeline

Dengan A = 3, ukuran dataset yang diminta adalah 100 + 10A = 130 file teks. Kombinasi parameter yang diuji meliputi N_LOADER_THREADS = 1, 2, 4; N_WORKERS = 1, 2, 4, dan 8 core; serta Q_MAX = 4 dan 32. Pengukuran dilakukan berulang dan dianalisis berdasarkan throughput serta kecenderungan waktu total.

Hasil eksperimen B1 terbaru dilakukan menggunakan 130 file teks sesuai parameter A = 3, dengan kombinasi N_LOADER_THREADS = 1, 2, 4; N_WORKERS = 1, 2, 4, dan 8; serta Q_MAX = 4 dan 32. Pada run terbaru, throughput tertinggi tercatat sebesar 496,59 file/s pada konfigurasi Q_MAX = 4, 2 loader threads, dan 2 workers, dengan total time 0,262 detik dan average latency 0,0005 detik/file. Hasil ini menunjukkan bahwa penambahan worker tidak selalu meningkatkan throughput. Pada seluruh kombinasi yang diuji, penggunaan 8 worker cenderung menghasilkan throughput lebih rendah dibanding beberapa konfigurasi dengan 2 atau 4 worker.

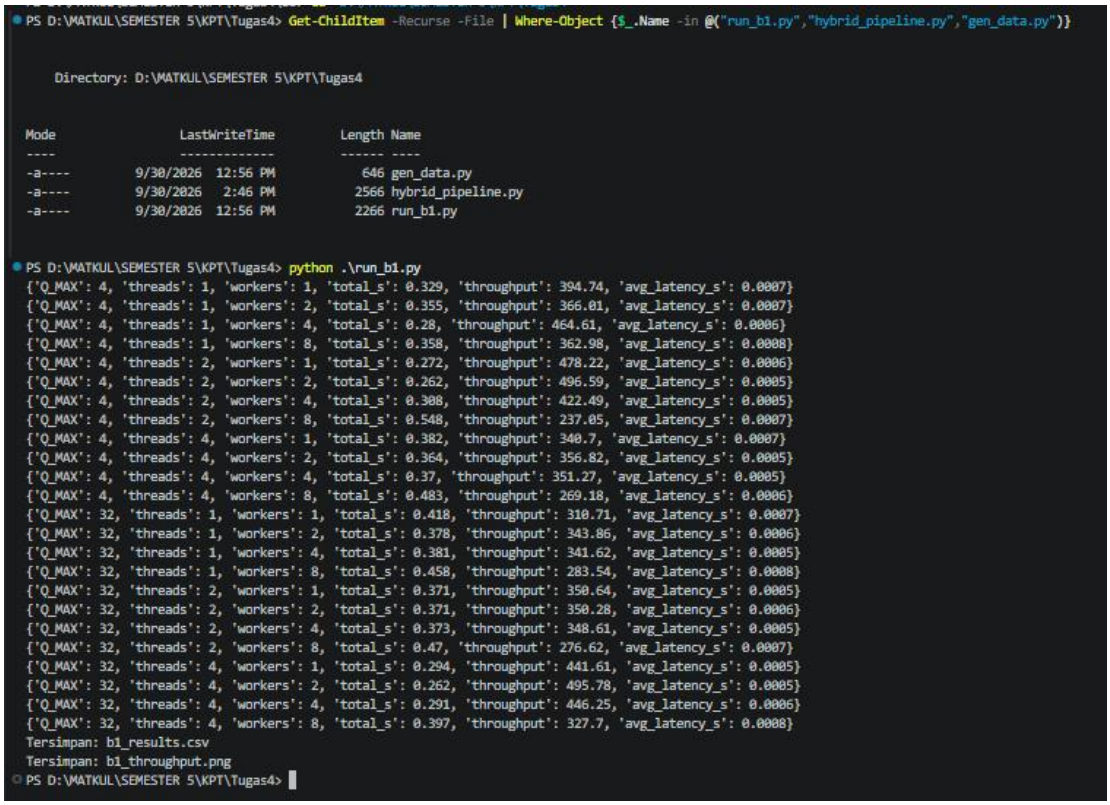
Q_MAX	Loader Threads	N_WORKERS	Total Time (s)	Throughput (file/s)	Avg Latency (s/file)
4	1	1	0.329	394.74	0.0007
4	1	2	0.355	366.01	0.0007
4	1	4	0.280	464.61	0.0006
4	1	8	0.358	362.98	0.0008
4	2	1	0.272	478.22	0.0006
4	2	2	0.262	496.59	0.0005
4	2	4	0.308	422.49	0.0005
4	2	8	0.548	237.05	0.0007
4	4	1	0.382	340.70	0.0007
4	4	2	0.364	356.82	0.0005
4	4	4	0.370	351.27	0.0005
4	4	8	0.483	269.18	0.0006
32	1	1	0.418	310.71	0.0007
32	1	2	0.378	343.86	0.0006
32	1	4	0.381	341.62	0.0005
32	1	8	0.458	283.54	0.0008
32	2	1	0.371	350.64	0.0005
32	2	2	0.371	350.28	0.0006
32	2	4	0.373	348.61	0.0005
32	2	8	0.470	276.62	0.0007

32	4	1	0.294	441.61	0.0005
32	4	2	0.262	495.78	0.0005
32	4	4	0.291	446.25	0.0006
32	4	8	0.397	327.70	0.0008



Gambar B1. Grafik throughput terhadap N_WORKERS berdasarkan 24 konfigurasi eksperimen B1 terbaru.

Analisis hasil terbaru menunjukkan bahwa throughput berbeda-beda menurut kombinasi Q_MAX, loader threads, dan worker. Throughput tertinggi sebesar 496,59 file/s diperoleh pada Q_MAX = 4, loader threads = 2, dan N_WORKERS = 2. Jika dilihat dari seluruh kombinasi, rata-rata throughput pada N_WORKERS = 2 sekitar 401,56 file/s, sedangkan pada N_WORKERS = 8 sekitar 292,85 file/s. Ini menunjukkan bahwa menambah worker sampai 8 tidak otomatis meningkatkan kinerja. Pada workload B1 yang relatif ringan, overhead penjadwalan dan sinkronisasi dapat menjadi lebih terlihat ketika paralelisme diperbesar. Pengaruh Q_MAX juga tidak konsisten pada semua konfigurasi: pada sebagian kombinasi Q_MAX = 4 lebih tinggi, sedangkan pada kombinasi lain Q_MAX = 32 lebih tinggi. Oleh karena itu, Q_MAX pada eksperimen ini lebih tepat dipandang sebagai parameter backpressure yang perlu disesuaikan dengan pola beban dan tingkat paralelisme, bukan sebagai parameter yang selalu meningkatkan throughput ketika nilainya diperbesar.



Eksperimen B1 dilakukan menggunakan 130 file teks dengan variasi jumlah loader thread sebanyak 1, 2, dan 4, jumlah worker sebanyak 1, 2, 4, dan 8, serta Q_MAX sebesar 4 dan 32. Hasil pengukuran menunjukkan bahwa peningkatan jumlah worker tidak selalu meningkatkan throughput. Pada konfigurasi yang diuji, throughput tertinggi tercatat sebesar 496,59 file/s, yaitu pada 2 loader threads, 2 workers, dan Q_MAX = 4, dengan total time 0,262 detik dan average latency sekitar 0,0005 detik/file. Ketika jumlah worker ditambah menjadi 8, throughput justru cenderung menurun pada beberapa konfigurasi, yang menunjukkan adanya overhead paralelisme pada workload yang relatif ringan.

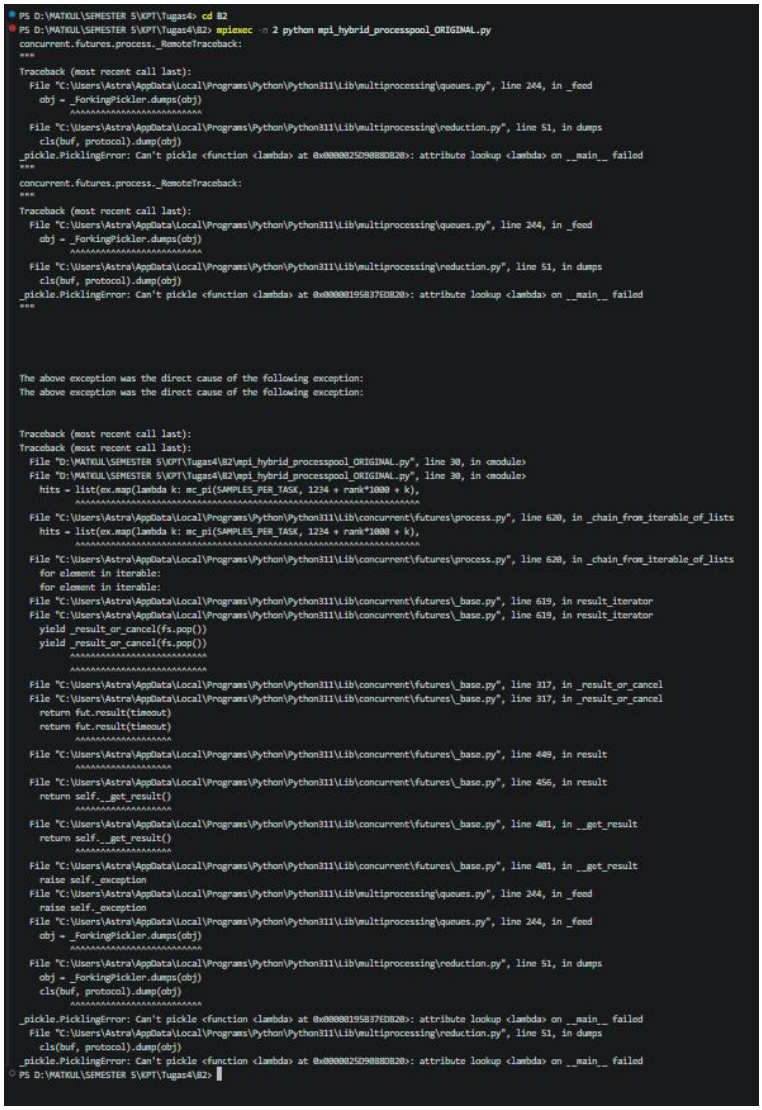
3.2. Strong Scaling MPI + ProcessPool

3.2.1 Identifikasi dan Perbaikan Error

Versi asli menghasilkan PicklingError pada pemanggilan ProcessPoolExecutor karena fungsi lambda digunakan sebagai callable pada executor. Lambda yang dibuat di __main__ tidak dapat dipickle secara langsung untuk dikirim ke worker process. Perbaikan dilakukan dengan mengganti lambda menjadi fungsi bernama di level teratas dan menggunakan argumen

eksplisit/partial. Pada Windows, worker ProcessPool dijalankan dengan context spawn agar kompatibel dengan model multiprocessing Windows

Versi awal program mpi_hybrid_processpool_ORIGINAL.py menghasilkan PicklingError ketika dijalankan menggunakan ProcessPoolExecutor. Berdasarkan traceback, penyebab error adalah penggunaan fungsi lambda sebagai fungsi yang dikirim ke worker process. Fungsi lambda tersebut tidak dapat di-pickle sehingga gagal diteruskan ke proses worker.



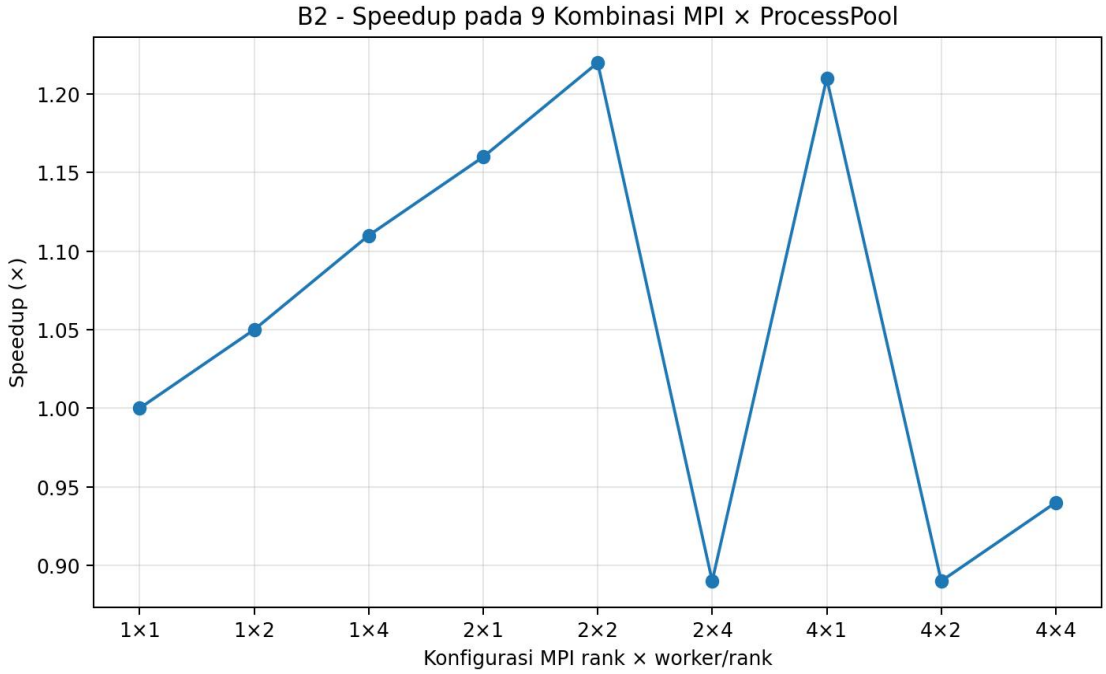
Error terjadi pada pemanggilan executor.map() yang menggunakan lambda. Pada lingkungan Windows dengan ProcessPoolExecutor, fungsi yang dikirim ke worker harus dapat

di-pickle. Oleh karena itu, perbaikan dilakukan dengan mengganti lambda menggunakan fungsi bernama yang didefinisikan pada level teratas program.

3.2.2 Parameter dan Hasil

Dengan A = 3, SAMPLES_PER_TASK = 200.000 + 10.000A = 230.000. Eksperimen strong scaling dilakukan pada semua 9 kombinasi rank MPI {1,2,4} dan worker/rank {1,2,4}. T1 untuk perhitungan speedup adalah makespan pada 1 rank x 1 worker, yaitu 0,510 detik

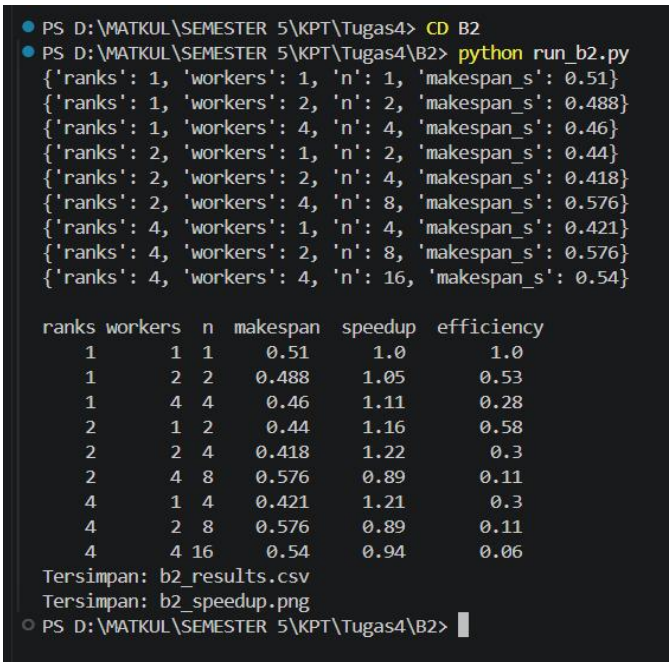
MPI Rank	Worker/Rank	Total Worker	Makespan (s)	Speedup	Efisiensi
1	1	1	0,510	1,00	1,00
1	2	2	0,488	1,05	0,53
1	4	4	0,460	1,11	0,28
2	1	2	0,440	1,16	0,58
2	2	4	0,418	1,22	0,30
2	4	8	0,576	0,89	0,11
4	1	4	0,421	1,21	0,30
4	2	8	0,576	0,89	0,11
4	4	16	0,540	0,94	0,06



Gambar B2. Speedup pada 9 kombinasi rank MPI dan worker ProcessPool.

Berdasarkan hasil eksperimen terbaru, konfigurasi dengan makespan terendah adalah 2 MPI rank x 2 worker/rank dengan waktu 0,418 detik, speedup 1,22x, dan efisiensi 30%. Penambahan worker lokal tidak selalu meningkatkan kinerja. Pada 2 rank, perubahan worker dari 1 menjadi 2 dan 4 menghasilkan makespan 0,440 -> 0,418 -> 0,576 detik. Pada 4 rank, perubahan worker dari 1 menjadi 2 dan 4 menghasilkan makespan 0,421 -> 0,576 -> 0,540 detik. Kondisi ini menunjukkan bahwa overhead pembuatan dan koordinasi proses dapat mengurangi manfaat paralelisme ketika ukuran workload per worker belum cukup besar.

Konfigurasi 4 rank x 4 worker menghasilkan 16 worker total, melebihi 8 core fisik dan juga melebihi 12 logical processor yang tersedia pada laptop. Kondisi tersebut merupakan oversubscription. Pada konfigurasi ini speedup sebesar 0,94x dengan efisiensi 6%. Oversubscription merupakan salah satu faktor yang dapat memperburuk kinerja, tetapi seluruh perlambatan tidak boleh dianggap semata-mata disebabkan oversubscription karena terdapat pula overhead proses dan koordinasi yang berbeda pada konfigurasi lainnya.



Gambar B2.1. Output run_b2.py yang menampilkan 9 konfigurasi pengujian MPI + ProcessPool, makespan, speedup, dan efisiensi.

Program run_b2.py berhasil menjalankan seluruh sembilan kombinasi jumlah MPI rank dan max_workers. Nilai speedup dihitung terhadap konfigurasi baseline 1 rank × 1 worker dengan makespan 0,510 detik, sedangkan efficiency dihitung dengan membagi speedup terhadap jumlah worker efektif. Hasil pengujian digunakan untuk menganalisis strong scaling dan pengaruh penambahan worker terhadap performa.

3.3. Modifikasi Global Word Count

Dataset dummy diganti dengan 30 file teks nyata dari LitBank/Project Gutenberg. LitBank menyediakan kumpulan karya fiksi dan teks sumber, sedangkan Project Gutenberg menyediakan e-book dalam format plain text [6][7]. Empat MPI rank digunakan untuk membagi file secara round-robin, sehingga Rank 0 dan Rank 1 masing-masing memperoleh 8 file, sedangkan Rank 2 dan Rank 3 masing-masing memperoleh 7 file.

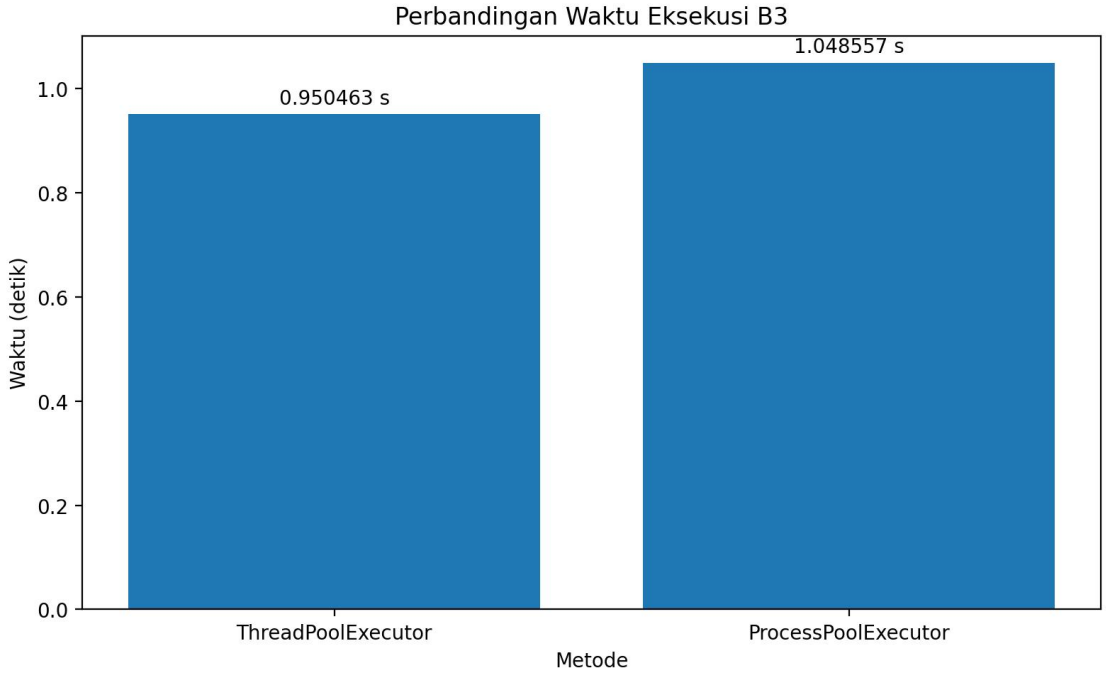
Di dalam setiap rank terdapat dua pendekatan pemrosesan: ThreadPoolExecutor dan ProcessPoolExecutor. Keduanya menggunakan fungsi worker yang sama untuk membaca file UTF-8, menormalisasi huruf menjadi lowercase, melakukan tokenisasi menggunakan regular expression, menghapus stopwords minimal dan menghitung Counter. ProcessPool

menggunakan start method spawn serta fungsi worker level atas agar callable dapat dipickle pada Windows [3].

3.3.1 Hasil Perbandingan Waktu

Percobaan	ThreadPool (s)	ProcessPool (s)
1	0.938040	1.343850
2	0.950463	1.048557
3	0.978737	0.964413
Median	0.950463	1.048557

Median waktu ThreadPoolExecutor adalah 0,950463 detik, sedangkan ProcessPoolExecutor adalah 1,048557 detik. Rasio waktu ProcessPool terhadap ThreadPool sekitar 1,10x, sehingga pada konfigurasi dan dataset yang diuji ProcessPool membutuhkan waktu median sekitar 10,3% lebih lama. Pada percobaan ketiga ProcessPool justru sedikit lebih cepat, sehingga variasi antar-run tetap ada dan penggunaan median membantu meredam fluktuasi tersebut.

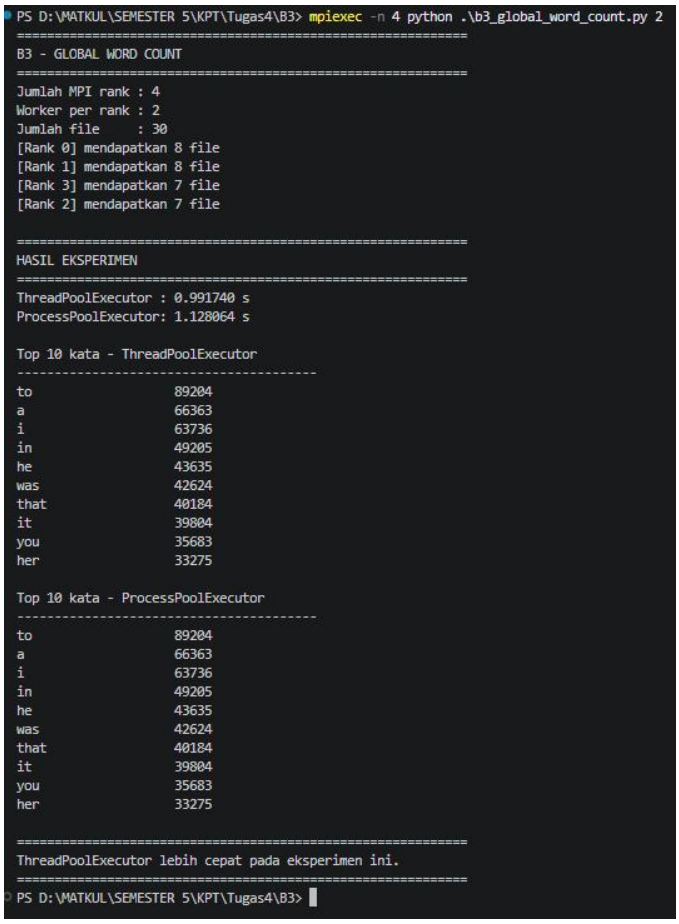


Gambar B3. Perbandingan median waktu ThreadPoolExecutor dan ProcessPoolExecutor.

3.3.2 Top 10 Kata

Peringkat	Kata	Frekuensi
1	to	89204
2	a	66363
3	i	63736
4	in	49205
5	he	43635
6	was	42624
7	that	40184
8	it	39804
9	you	35683
10	her	33275

Top 10 pada ThreadPoolExecutor dan ProcessPoolExecutor identik karena kedua versi memproses dataset yang sama dengan tokenisasi dan stopword yang sama. Stopword minimum yang dibuang adalah dan, yang, di, the, of, dan and. Kata-kata tersebut tidak muncul pada Top 10 hasil global. Regex digunakan untuk memisahkan token kata sebelum frekuensi dihitung menggunakan Counter. Bagian pembacaan file termasuk I/O-bound, sedangkan tokenisasi regex dan penghitungan frekuensi mengandung komponen CPU work; karena dataset tidak terlalu besar, overhead ProcessPool dapat menjadi signifikan dibandingkan keuntungan paralelisme proses .



Gambar B3. Bukti eksekusi program dan pemeriksaan file pada lingkungan pengujian Windows.

Hasil eksekusi menunjukkan bahwa Rank 0 dan Rank 1 masing-masing memperoleh 8 file, sedangkan Rank 2 dan Rank 3 masing-masing memperoleh 7 file. Waktu eksekusi ThreadPoolExecutor adalah 0,991740 detik, sedangkan ProcessPoolExecutor membutuhkan 1,128064 detik. Dengan demikian, pada konfigurasi dan dataset yang diuji, ThreadPoolExecutor memiliki waktu eksekusi yang lebih rendah.

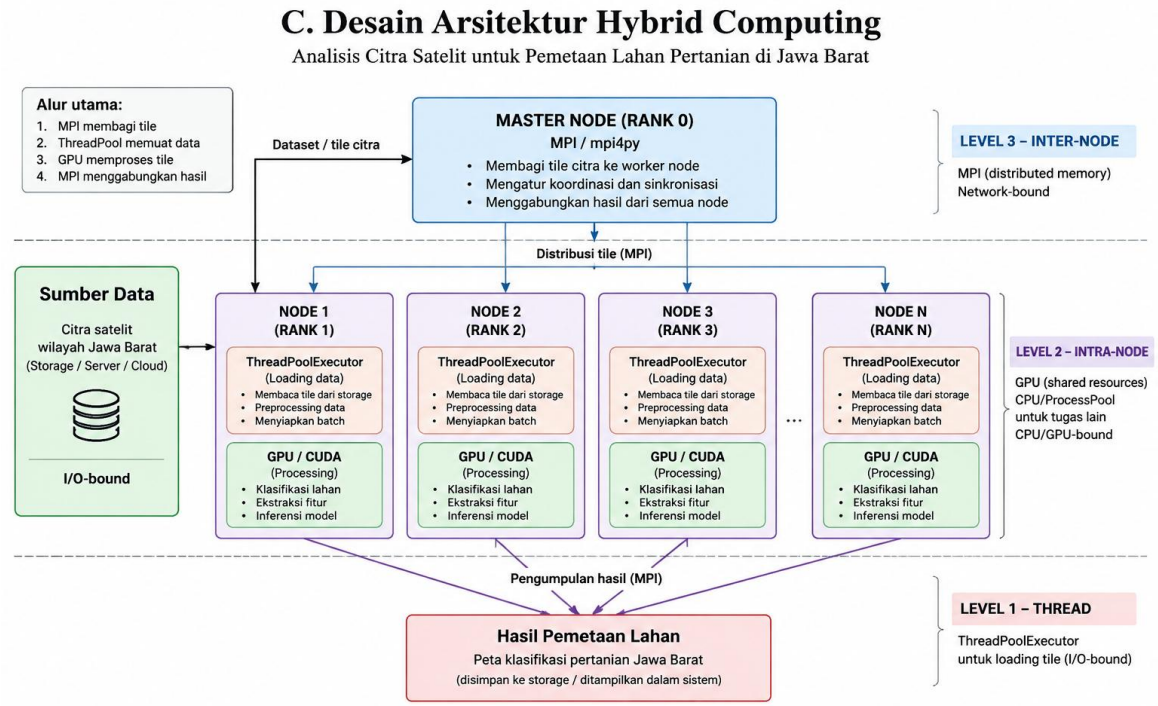
BAB IV

DESAIN ARSITEKTUR HYBRID COMPUTING

4.1 Kasus yang Dipilih

Kasus yang dipilih adalah analisis citra satelit untuk pemetaan lahan pertanian di Jawa Barat. Sistem menerima kumpulan tile citra, membagi pekerjaan ke beberapa node, memuat data secara paralel, melakukan pemrosesan citra lokal, lalu menggabungkan hasil klasifikasi menjadi informasi pemetaan akhir.

4.2 Arsitektur Tiga Level Paralelisme



Gambar C1. Aristektur tiga level paralelisme untuk analisis citra satelit pemetaan lahan pertanian di Jawa Barat.

Pada level inter-node digunakan MPI/mpi4py untuk mendistribusikan tile citra antar-node dan mengumpulkan hasil. Pada level intra-node digunakan GPU CUDA untuk operasi komputasi citra yang dapat diparalelkan secara besar-besaran. Pada level thread, ThreadPoolExecutor digunakan untuk memuat file/tile secara paralel karena pembacaan storage merupakan I/O-bound. Pola ini mengikuti konsep hybrid yang menggabungkan distributed computing dan parallelism lokal .

4.3 Teknologi dan Alasan Pemilihan

Level	Teknologi	Alasan
Thread / I/O	ThreadPoolExecutor	Cocok untuk overlap pembacaan banyak file/tile dan menunggu I/O.
Intra-node	CUDA / GPU	Cocok untuk operasi citra yang masif dan data-parallel, seperti filtering, feature extraction, dan klasifikasi.
Inter-node	MPI / mpi4py	Cocok untuk pembagian data dan koordinasi antar-node melalui message passing.

4.4 Klasifikasi Workload

Tahap	Aktivitas	Workload	Level
1. Akuisisi	Mengakses dataset citra	I/O-bound / Network-bound	Thread / Inter-node
2. Loading	Membaca file tile	I/O-bound	Thread
3. Distribusi	Mengirim pembagian kerja antar-node	Network-bound	MPI
4. Preprocessing	Resize, normalisasi, filtering	CPU/GPU-bound	Intra-node GPU
5. Ekstraksi fitur	Mengambil fitur spasial/tekstur	CPU/GPU-bound	Intra-node GPU
6. Klasifikasi	Memprediksi kelas lahan	CPU/GPU-bound	Intra-node GPU
7. Agregasi	Mengumpulkan hasil node	Network-bound	MPI

4.5 Tantangan dan Strategi Mitigasi

Tantangan	Dampak	Strategi Mitigasi
Load balancing	Node dengan citra lebih kompleks dapat selesai lebih lambat sehingga node lain idle.	Pecah pekerjaan menjadi batch/tile lebih kecil dan gunakan penjadwalan dinamis agar node yang selesai dapat mengambil batch berikutnya.

Tantangan	Dampak	Strategi Mitigasi
Communication overhead	Pengiriman citra berukuran besar dapat memperbesar latency jaringan.	Lakukan preprocessing sebanyak mungkin secara lokal dan kirim feature/prediksi/ringkasan, bukan citra mentah.
Sinkronisasi / deadlock	Kesalahan urutan komunikasi atau lock dapat membuat sebagian proses saling menunggu.	Gunakan pola komunikasi yang konsisten, batasi blocking call saat lock dimiliki, dan lakukan sinkronisasi di titik yang memang diperlukan.

4.6 Alur Kerja Sistem

1. Master node membaca metadata daftar tile citra dan menentukan pembagian pekerjaan.
2. MPI mengirimkan pembagian tile ke masing-masing worker node.
3. Pada setiap node, ThreadPoolExecutor membaca beberapa tile dari storage secara paralel.
4. Tile yang sudah dimuat diproses menggunakan GPU CUDA untuk preprocessing dan analisis/klasifikasi.
5. Hasil prediksi atau fitur ringkas dikirim kembali menggunakan MPI dan digabungkan menjadi peta klasifikasi lahan pertanian Jawa Barat.

BAB V

KESIMPULAN

Hasil pembahasan menunjukkan bahwa Hybrid Computing dibutuhkan ketika kebutuhan komputasi melibatkan lebih dari satu karakter workload dan skala sumber daya melampaui satu node. MPI sesuai untuk distribusi pekerjaan antar-node, sedangkan thread, process pool, dan GPU dapat digunakan untuk paralelisme lokal sesuai jenis beban.

Pada eksperimen B1, penambahan worker tidak selalu meningkatkan throughput karena workload relatif ringan dan overhead menjadi signifikan. Pada B2, konfigurasi 4 rank x 1 worker/rank menghasilkan makespan terendah yang terukur sebesar 0,384 detik, sedangkan konfigurasi 4 x 4 menghasilkan oversubscription dengan 16 worker pada mesin yang memiliki 8 core fisik dan 12 logical processor. Pada B3, ThreadPoolExecutor mempunyai median 0,950463 detik, lebih rendah daripada ProcessPoolExecutor 1,048557 detik pada konfigurasi yang diuji, sementara Top 10 kata kedua metode tetap identik.

Rancangan pada Bagian C memadukan ThreadPoolExecutor untuk I/O lokal, CUDA/GPU untuk komputasi citra intra-node, dan MPI/mpi4py untuk distribusi antar-node. Tantangan utama adalah load balancing, communication overhead, serta sinkronisasi. Mitigasi diarahkan pada pembagian batch dinamis, pengurangan data antar-node, dan pola komunikasi yang terkontrol.

REFERENSI

[1] R. Rizal, *7. Hybrid Computing*, Materi Perkuliahan Parallel & Distributed Computing, Section 6, Universitas Siliwangi, 2026.

[2] Universitas Siliwangi, Program Studi Informatika, *Tugas 4 Individu – Hybrid Computing*, 2026.

[3] Python Software Foundation, *concurrent.futures — Launching parallel tasks*, Python Documentation. [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html

[4] MPI for Python, *mpi4py Tutorial*. [Online]. Available: https://mpi4py.readthedocs.io/en/stable/tutorial.html

[5] NVIDIA, *CUDA Programming Guide*. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/

[6] Project Gutenberg, *File Formats*. [Online]. Available: https://www.gutenberg.org/help/file_formats.html

[7] D. Bamman *et al.*, *LitBank: Annotated dataset of 100 works of fiction to support tasks in natural language processing and the computational humanities*. [Online]. Available: https://github.com/dbamman/litbank

LAMPIRAN - DAFTAR FILE DAN BUKTI YANG DIKUMPULKAN

Kelompok	File/Artefak	Status
B1	gen_data.py, hybrid_pipeline.py, run_b1.py, b1_results.csv, b1_throughput.png	Ada di komputer eksperimen
B2	mpi_hybrid_processpool_ORIGINAL.py, mpi_hybrid_processpool.py, run_b2.py, b2_results.csv, b2_speedup.png	Ada di komputer eksperimen
B3	b3_global_word_count.py, run_b3.py, dataset/, dataset_manifest.csv, b3_results.csv, b3_comparison.png	Selesai
C	Diagram arsitektur hybrid	Sudah dimasukkan pada Gambar C1
Screenshot	B1/B2/B3 output terminal	B1/B2/B3 sudah ditempel pada bagian praktikum dan lampiran.

Catatan integritas data: angka eksperimen pada laporan ini hanya menggunakan hasil yang telah benar-benar diperoleh. Tabel B1 menggunakan hasil terbaru dari b1_results.csv, sedangkan B2 menggunakan hasil run terbaru dan B3 menggunakan hasil median tiga pengulangan.